Indian Institute of Technology, Bhilai

CSP203: Software Tools & Technologies Laboratory

Lab Activity - 19

Full Marks: 55 *Deadline: 11:59 PM, 21-11-2025*

Submission Instructions:

1) Answer all the questions.
2) You should prepare a report which contains screenshots of code and output of the task. The report contains **5 marks.**
3) Your report file should be named as <Roll No>_<Name>_<Lab Activity No>.pdf
4) Example: B24CS055_Shareen_18.pdf
5) **Plagiarism is strictly prohibited.**

Objectives:

1) To understand and configure Docker user-defined bridge networks.
2) To gain hands-on experience in container-level networking diagnostics.
3) To develop proficiency in using GDB for interactive debugging of C programs.
4) To identify and resolve logical and memory-related bugs in C code.
5) To build systematic debugging and analysis skills through breakpoints, backtrace, and variable tracking.

Question - 1: [20 Marks]

Single Bridge Network, 10 Ubuntu Containers

Part 1:

Create a Docker **bridge network** named:

mynet

The network must have:

1. Subnet: 182.18.0.0/16
2. Gateway: 182.18.0.1
3. Default **bridge** driver

Part 2:

Create **10 containers** using ONLY the **Ubuntu image**:

ubuntu1, ubuntu2, ubuntu3, ubuntu4, ubuntu5, ubuntu6, ubuntu7, ubuntu8, ubuntu9, ubuntu10

Requirements:

1. All containers must connect to **mynet** at creation time.
2. Assign **static IP addresses** to **ubuntu1** and **ubuntu2** using --ip.
3. Allow the other containers to obtain IP addresses **automatically** from Docker.
4. All containers must start with **bash** as the command.

Part 3:

Inside **ubuntu1** and **ubuntu5**, install ping.

Part 4:

- **Part A — DNS-based Connectivity:**
  From **ubuntu1**, ping **four** other containers using **container names only**

(e.g., ping ubuntu4).
This test verifies Docker's **embedded DNS**.

- **Part B — IP-based Connectivity:**
  From **ubuntu5**, ping any **four** containers using their **IP addresses**.

Part 5:

Using the output of inspect command, answer the following:

1. What IP address was assigned to each of the 10 Ubuntu containers?
2. What is the **gateway** of the mynet network?
3. What **MAC address** does ubuntu1 have?
4. What does the "Containers" section describe in the inspect output?
5. What DNS server is used for resolving container names?

Part 6:

Perform the following:

1. Disconnect **ubuntu10** from the network mynet.
2. Reconnect **ubuntu10** back to the network mynet.

Submit commands and screenshots for each step.

Question - 2: [20 Marks]

Source File: lab_exam.c

```
#include <stdio.h>
int sum_array(int arr[], int size) {
   int sum = 0;
   for(int i = 0; i < size; i++) {
      sum += arr[i];
   }
```

```
    return sum;
}
int max_element(int arr[], int size) {
    int max = arr[0];
    for(int i = 1; i < size; i++) {
        if(arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
int main() {
    int data[] = {4, 8, 15, 16, 23, 42};
    int n = sizeof(data) / sizeof(data[0]);

    int total = sum_array(data, n);
    printf("Sum: %d\n", total);

    int maximum = max_element(data, n);
    printf("Max: %d\n", maximum);

    return 0;
}
```

Step 1: Compilation

1. Compile the program with debugging symbols.
2. What command did you use?
3. How can you verify the binary includes debug information?

Step 2: Start GDB and Initial Setup

1. Launch GDB with the compiled binary.
2. List the contents of the main() function.
3. Set a breakpoint at the start of main() and run the program until the breakpoint is hit.

Step 3: Navigating Through Code

1. Use next to step over the first three lines inside main().

2. Use step to enter into the sum_array function.
3. Step through the loop. What are the values of sum and arr[i] at each iteration?

Step 4: Variable Inspection

1. Print the contents of data using a loop in GDB.
2. Print the values of n, total, and maximum.

Step 5: Watchpoints

1. Set a watch on the variable sum during sum_array() execution.
2. Observe how its value changes step-by-step.
3. Set a conditional breakpoint to stop when arr[i] == 16.

Step 6: Function-Specific Breakpoints

1. Set a breakpoint at the max_element function using its name.
2. Run the program and ensure it stops at the correct point.
3. Step into the function and print the value of max after each comparison.

Step 7: Call Stack and Frame Navigation

1. Inside max_element, use backtrace to show the call stack.
2. Use frame, up, and down to navigate through frames.

Step 8: Modifying Variables

1. Modify the first element of the array (data[0]) from 4 to 100 in GDB.
2. Re-run sum_array() manually from GDB and observe the new output.

Step 9: Forced Return

1. Force sum_array() to return 999 using the return command in GDB.
2. What is the printed value for the sum now?

Step 10: Bonus (Advanced)

1. Set a breakpoint at printf using the function name.
2. Step into printf. What function do you enter?
3. How can you avoid stepping into external functions?

Provide commands and screenshots.

Question - 3: [10 Marks]

Problem Description:

You are given a C program that manages a dynamic integer array with the following features:

- Initialize the array with a fixed capacity.
- Insert integers at specified indices.
- Remove integers at specified indices.
- Print the array contents.

The program sometimes **crashes** or **prints corrupted data** after a series of insertions and removals.

Your tasks:

- Use **GDB** to debug the program:
  - Set **breakpoints (b)** at key functions (insert, remove, main).
  - Use **watchpoints** to monitor changes to the array or important variables.
  - Use **bt (backtrace)** when the program crashes to identify where the error happened.
- Identify the cause of the crash or corruption.
- Fix the program to avoid memory corruption.
- Confirm the fix by running the program multiple times without crash or corruption.

Provided Code (dynamic_array_bug.c)

```
#include <stdio.h>

#include <stdlib.h>

#define CAPACITY 5

typedef struct {

    int* data;

    int size;
```

```
    int capacity;
} DynamicArray;
void initArray(DynamicArray* arr) {
    arr->data = malloc(sizeof(int) * CAPACITY);
    arr->size = 0;
    arr->capacity = CAPACITY;
}
void insert(DynamicArray* arr, int index, int value) {
    if (index < 0 || index > arr->size || arr->size == arr->capacity) {
        printf("Insert Error: Invalid index or array full\n");
        return;
    }
    // Shift elements right to make space
    for (int i = arr->size; i >= index; i--) {
        arr->data[i] = arr->data[i-1];  // <-- Bug here: i == arr->size, i-1 might be invalid!
    }
    arr->data[index] = value;
    arr->size++;
}
void removeAt(DynamicArray* arr, int index) {
    if (index < 0 || index >= arr->size) {
        printf("Remove Error: Invalid index\n");
```

```
        return;
    }
    for (int i = index; i < arr->size - 1; i++) {
        arr->data[i] = arr->data[i+1];
    }
    arr->size--;
}
void printArray(DynamicArray* arr) {
    printf("Array contents: ");
    for (int i = 0; i < arr->size; i++) {
        printf("%d ", arr->data[i]);
    }
    printf("\n");
}
int main() {
    DynamicArray arr;
    initArray(&arr);
    insert(&arr, 0, 10);
    insert(&arr, 1, 20);
    insert(&arr, 2, 30);
    insert(&arr, 1, 15);
    insert(&arr, 4, 40);
```

```
    printArray(&arr);
    removeAt(&arr, 2);
    printArray(&arr);

    insert(&arr, 2, 25);
    printArray(&arr);
    insert(&arr, 5, 50);
    insert(&arr, -1, 60);
    free(arr.data);
    return 0;
}
```

Your Tasks:

1. Compile with debugging symbols.
2. Run the program. It may crash or show corrupted output.
3. Use GDB to:
    - Set breakpoints at insert and removeAt functions.
    - Set a **watchpoint** on arr->size or arr->data to observe unexpected changes.
    - Step through the buggy insertions.
    - When the program crashes, use bt to view the call stack and locate the faulty line.
4. Identify the bug in insert related to the array indexing during the shift operation.
5. Fix the bug so shifting elements is done correctly without invalid memory access.
6. Re-run and verify that no crash occurs and the output is correct.

Expected Deliverables:

- Fixed code with comments explaining the changes.
- A GDB debugging session screenshots showing:

    - Breakpoints being hit.
    - Watchpoint triggering.
    - Backtrace output after the crash.
    - Variable inspections showing the bug cause.
    - Program code and output after debugging the error.
- A brief explanation of your debugging strategy using b, watch, and bt.